Name: **SOLUTION** UID#: DirectoryID:

CMSC216: Practice Final Exam B SOLUTION

Spring 2026, University of Maryland

Exam period: 20 minutes *Points available:* 40

Problem 1 (20 pts): Nearby is a serial function called `array_average()`. Write a Threaded version of this function that does not use any global variables. For full credit, utilize a mutex to coordinate change to shared data.

```
float array_average_serial(float *arr, int len){
  float sum = 0.0;
  for(int i=0; i<len; i++){
    sum += arr[i];
  }
  return sum / len;
}
```

```
////////////////////// SOLUTION //////////////////////
#include "array_average.h"
typedef struct {
  int thread_id;
  int num_threads;
  float *arr;
  int len;
  pthread_mutex_t *lock;
  float *total_sum;
} context_t;

void *workfunc(void *arg){
  context_t ctx = *((context_t*) arg);
  int count = (ctx.len / ctx.num_threads);
  int start = ctx.thread_id*count;
  int stop  = (ctx.thread_id+1)*count;
  if(ctx.thread_id == ctx.num_threads-1){
    stop = ctx.len;
  }
  double my_sum = 0.0;
  for(int i=start; i<stop; i++){
    my_sum += ctx.arr[i];
  }
  pthread_mutex_lock(ctx.lock);
  *ctx.total_sum += my_sum;
  pthread_mutex_unlock(ctx.lock);
  return NULL;
}

```

```
float array_average_threaded(float *arr, int len,
                             int num_threads)
{
  pthread_mutex_t lock;
  pthread_mutex_init(&lock,NULL);
  float total_sum = 0.0;
  pthread_t threads[num_threads];
  context_t context[num_threads];
  for(int i=0; i<num_threads; i++){
    context[i].thread_id = i;
    context[i].num_threads = num_threads;
    context[i].arr = arr;
    context[i].len = len;
    context[i].lock = &lock;
    context[i].total_sum = &total_sum;
    pthread_create(&threads[i],NULL,
                   workfunc, &context[i]);
  }
  for(int i=0; i<num_threads; i++){
    pthread_join(threads[i], (void **) NULL);
  }
  return total_sum / len;
}
```

Problem 2 (20 pts): Below are two functions that augment El Malloc with the block shrinking; this allows a user to specify that the originally requested size for a memory area can be adjusted down potentially creating open space. Fill in the definitions for these functions.

```
#include "el_malloc.c"
////////////////////////////// SOLUTION //////////////////////////////
el_blockhead_t *el_shrink_block(el_blockhead_t *head, size_t newsize){
// Shrinks the size of the given block potentially creating a new block.  Computes remaining space
// as the difference between the current size and parameter newsize. If this is smaller than
// EL_BLOCK_OVERHEAD, does nothing further and returns NULL. Otherwise, reduces the size of the
// given block by adjusting its header and footer and establishes a new block above it with
// remaining space beyond the block overhead. Returns a pointer to the newly introduced blocks. Does
// not modify any links in lists.

  // NOTE: could simplify considerably using el_split_block()
  size_t remaining = head->size - newsize;
  if(remaining < EL_BLOCK_OVERHEAD){
    return NULL;
  }
  head->size = newsize;                       // adjust size
  el_blockfoot_t *foot = el_get_footer(head); // allows middle foot to be found
  foot->size = newsize;                       // set

  el_blockhead_t *above_head = el_block_above(head); // new header location
  above_head->size = remaining - EL_BLOCK_OVERHEAD;  // set its size
  el_blockfoot_t *above_foot = el_get_footer(above_head); // should be old foot
  above_foot->size = remaining - EL_BLOCK_OVERHEAD;       // set size
  return above_head;
}

int el_shrink(void *ptr, size_t newsize){
// Shrink the area associated with the given ptr if possible.  Checks to ensure that the block
// associated with the given user ptr is EL_USED and exits if not.  Uses el_shrink_block() to
// adjust the block size and create a block for the remaining space.  If not possible to shrink,
// returns 0.  Otherwise moves the current block to the front of the Used List and places the newly
// created block to the front of the Available List after setting its state to EL_AVAILABLE. Returns
// 1 on successfully shrinking.

  el_blockhead_t *head = PTR_MINUS_BYTES(ptr,sizeof(el_blockhead_t));
  if(head->state != EL_USED){              // error check
    printf("Does not appear to be a used block\n");
    exit(1);
  }
  el_blockhead_t *above = el_shrink_block(head, newsize);
  if(above == NULL){
    return 0;                              // could not shrink
  }
  above->state = EL_AVAILABLE;             // now available for use
  el_remove_block(el_ctl->used, head);     // out of used list
  el_add_block_front(el_ctl->used,  head); // to front of used list
  el_add_block_front(el_ctl->avail, above); // to front of available list
  return 1;                                // could shrink
  // likely want to attempt merging above with block above to limit fragmentation
  // in a full implementation of shrinking
}
```